

PROJECT REPORT

Journal-entry testing, decided.

JE Agent — Journal Entry Testing AI Agent
per ISA 240 / AS 2401

Journal Entry Testing, Decided.

JE Agent — an AI-assisted audit assistant for journal entries (ISA 240 / AS 2401)

This report explains **what JE Agent is, how it thinks, and what the evidence says about how well it works** — without requiring a technical background. Deep technical detail lives in the repository (`DESIGN.md`, `PROJECT_REPORT.md`); this document focuses on the logic and the proof.

Contents

1. **The Problem** — why journal-entry testing needs help
 2. **The Logic** — three layers, arranged by trust
 3. **The Console** — a tour of the working environment
 4. **Does It Work?** — the stress test and what it proved
 5. **Trust & Integrity** — how the audit trail protects itself
 6. **Honest Limitations** — what JE Agent is not
 7. **Conclusion**
- Appendix A · The ten rules in plain language
 - Appendix B · Running it yourself

- Glossary

1 • The Problem

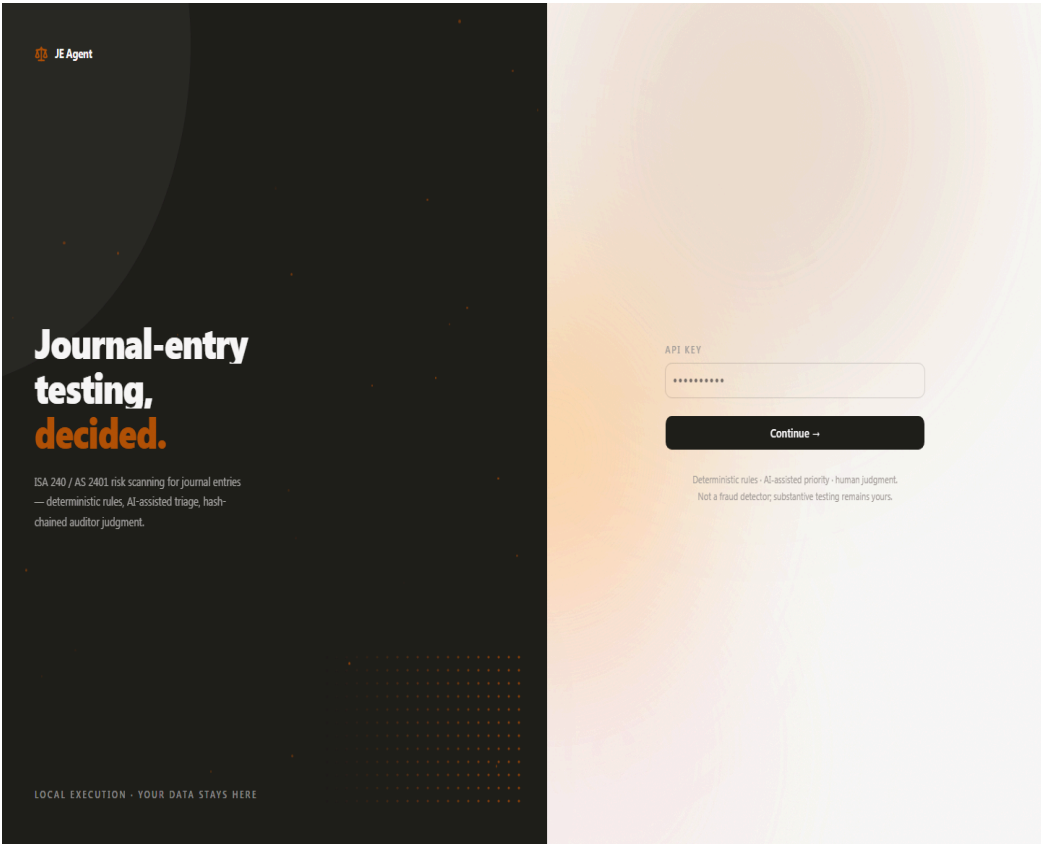
Auditors are required to test journal entries for fraud risk (ISA 240 / AS 2401). In practice this means reading through thousands — sometimes hundreds of thousands — of accounting lines, looking for the small number of entries that deserve suspicion:

- amounts posted at round numbers,
- invoices split into pieces to stay under approval limits,
- entries posted at unusual times (period end),
- unusual user activity, unusual account combinations,
- entries that don't balance, reversals, date inconsistencies...

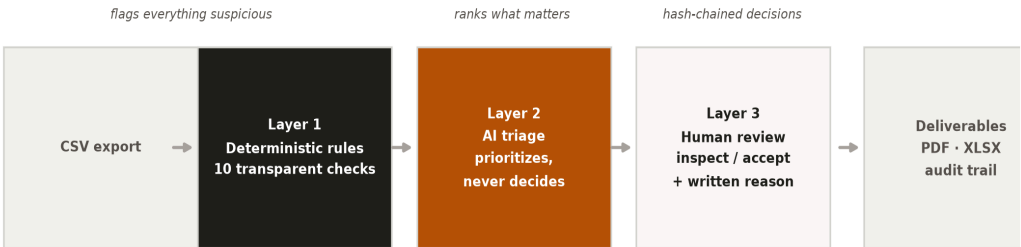
Doing this by hand is slow and unreliable. Doing it with a plain script produces thousands of “hits” that are almost all harmless. And doing it with AI alone raises the question every auditor must ask: *can I trust a black box?*

JE Agent’s answer: don’t choose between rules, AI, and humans — use all three, each where it is strongest.

2 · The Logic — Three Layers



JE Agent processes an accounting export through three layers, arranged exactly by trustworthiness:



Layer 1 — Deterministic rules (never wrong about *what it measures*). Ten transparent rules scan every entry: round amounts, splitting, period-end timing, balance checks, rare users, odd account pairs, reversals, date divergence, high-risk users, and manual-entry concentration. Every flag comes with its reason. These rules are fast and explainable — but as we will show in §4, they also over-flag: they are a *net*, not a verdict.

Layer 2 — AI triage (prioritization, never decision). An AI model reads the flagged entries and ranks which ones look most worth a human’s attention, writing a short rationale for each. Crucially, the AI **cannot finalize anything**: its output is advisory, recorded as such in the audit trail.

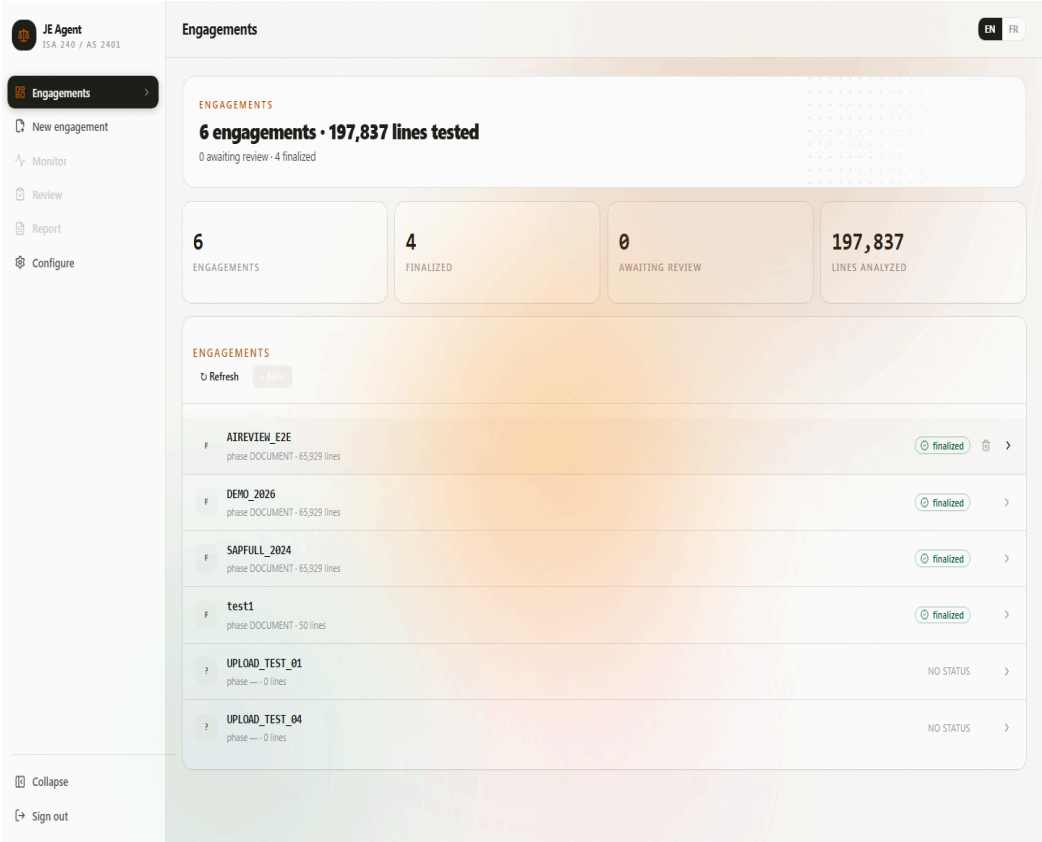
Layer 3 — Human judgment (where precision is made). Every entry that reaches review requires an explicit auditor decision — *inspect* or *accept* — with a mandatory written reason. Each decision is **hash-chained** to the previous one, so any later tampering would be mathematically detectable.

The pipeline then produces the deliverables an engagement file needs: a professional PDF report (in English or French), an Excel workpaper, and a list of flagged entries.

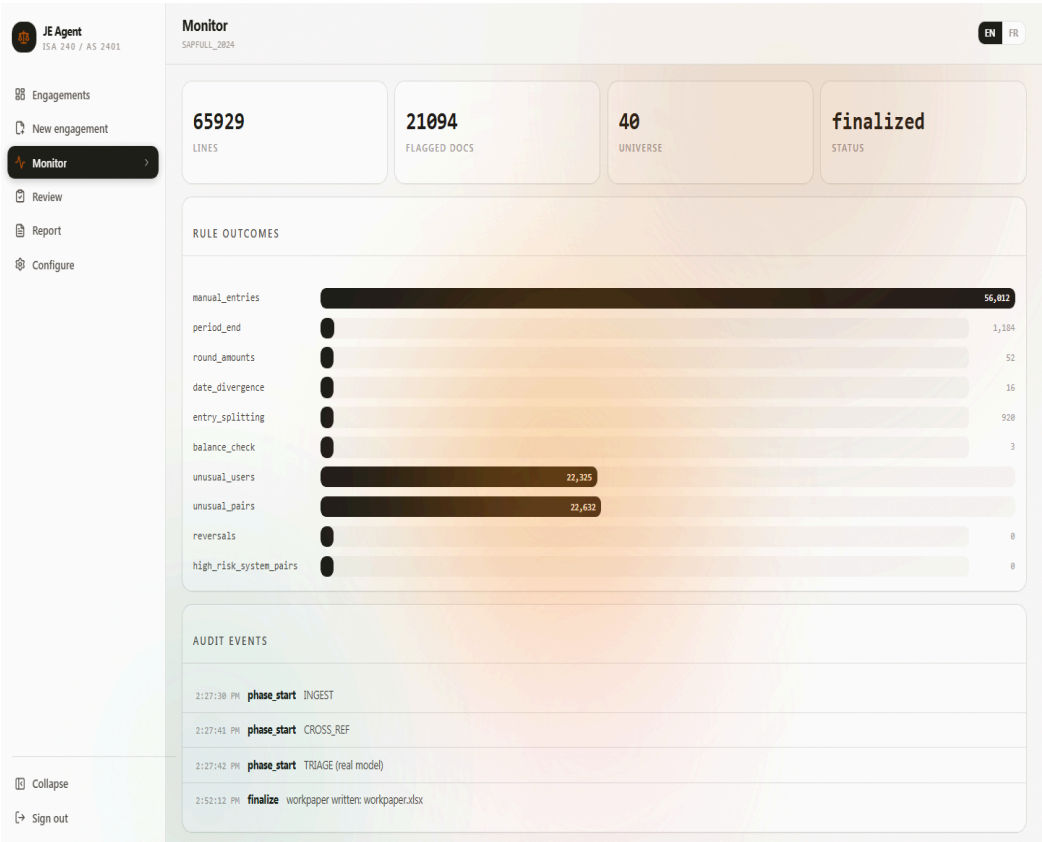
Design principle: the machine proposes, the human disposes. Nothing reaches “final” without a named person’s recorded judgment.

3 • The Console

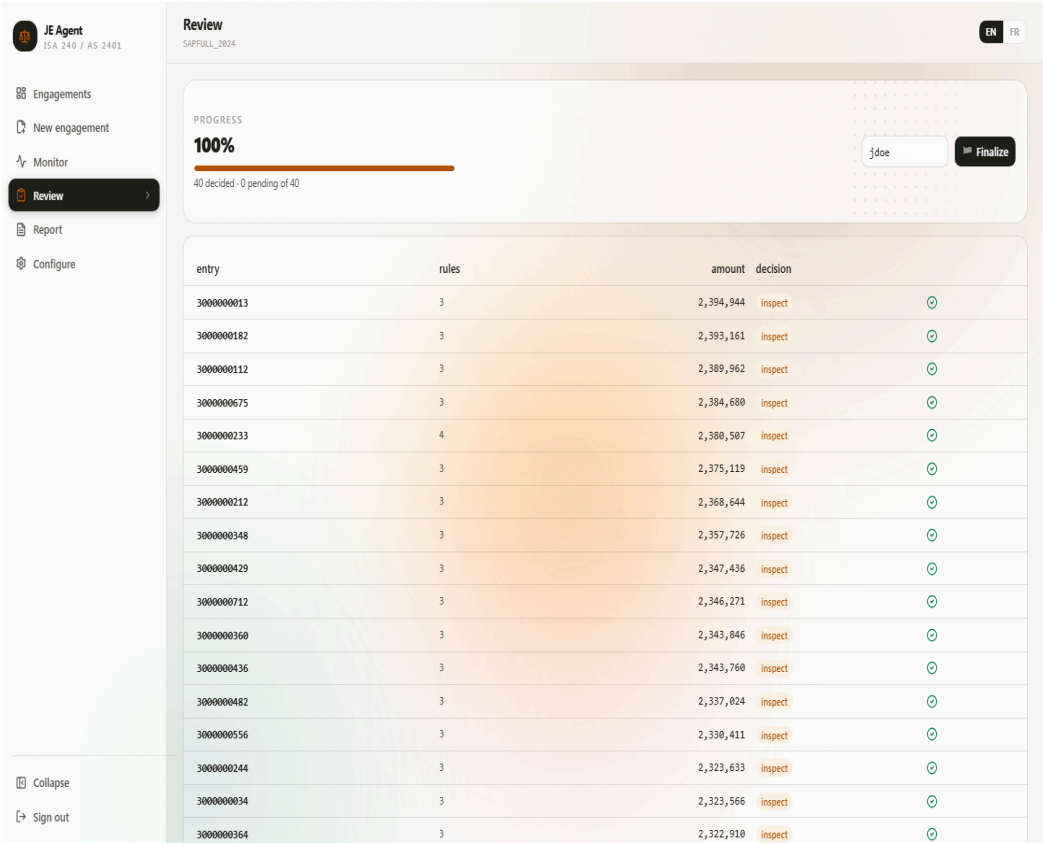
JE Agent runs locally — data never leaves the machine except optional AI triage calls (which run in zero-retention mode).



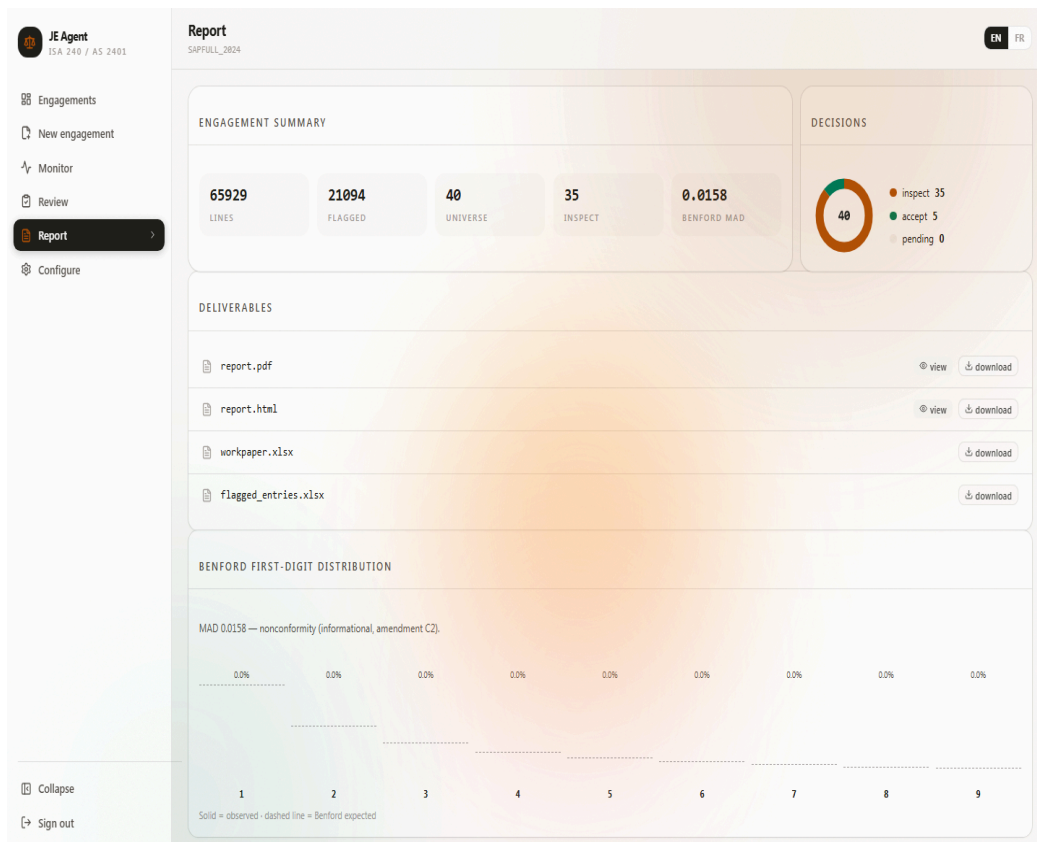
The dashboard summarizes every engagement: population sizes, finalization status, and one-click access to any run.



The Monitor page shows what each rule found and the full audit event timeline of the run.



The Review queue presents entries like a ledger. Clicking an entry opens its full journal lines and explains *which rules fired*. Decisions require a written reason and are hash-chained on confirmation.



The Report page gathers deliverables (PDF, HTML, Excel workpaper), a decisions breakdown, and a Benford first-digit analysis against the theoretically expected distribution.

JE Agent
ISA 240 / AS 2401

Engagements

New engagement

Monitor

Review

Report

Configure

Collapse

Sign out

New engagement

Drop your journal-entry CSV here
or click to browse · columns auto-detected

CONFIGURATION

ENGAGEMENT

Run ID
DEMO_2026

Period end
06/30/2026

Auto-detect from CSV

MATERIALITY

Overall
250000

Performance
175000

Currency
USD

SOURCE SYSTEM

System
SAP

Amount column
DMBTR

Currency column
WAERS

COLUMN MAPPING

Posting date
BUDAT

Document date
BLDAT

Account
HKONT

Username
UNAME

Description
SGTXT

Source doc
BELNR

Entry ref

Created date

Entry type

Creating an engagement is a single CSV upload — column mapping is auto-detected from the file itself, materiality and review mode are set in one form.

JE Agent
ISA 240 / AS 2401

Engagements

Nouvel engagement

Surveillance

Révision

Rapport

Configuration

Collapse

Déconnexion

Engagements

ENGAGEMENTS

6 engagements · 197,837 lines tested

0 awaiting review · 4 finalized

6
ENGAGEMENTS

4
FINALISÉS

0
EN ATTENTE DE RÉVISION

197,837
LIGNES ANALYSÉES

ENGAGEMENTS

Actualiser

6 engagements

F	AIREVIEW_E2E	phase DOCUMENT - 65,929 lines	finalized	>
F	DEMO_2026	phase DOCUMENT - 65,929 lines	finalized	>
F	SAPFULL_2024	phase DOCUMENT - 65,929 lines	finalized	>
F	test1	phase DOCUMENT - 50 lines	finalized	>
?	UPLOAD_TEST_01	phase — 0 lines	NO STATUS	>
?	UPLOAD_TEST_04	phase — 0 lines	NO STATUS	>

The entire interface — and the generated report — switch between English and French.

4 • Does It Work? The Stress Test

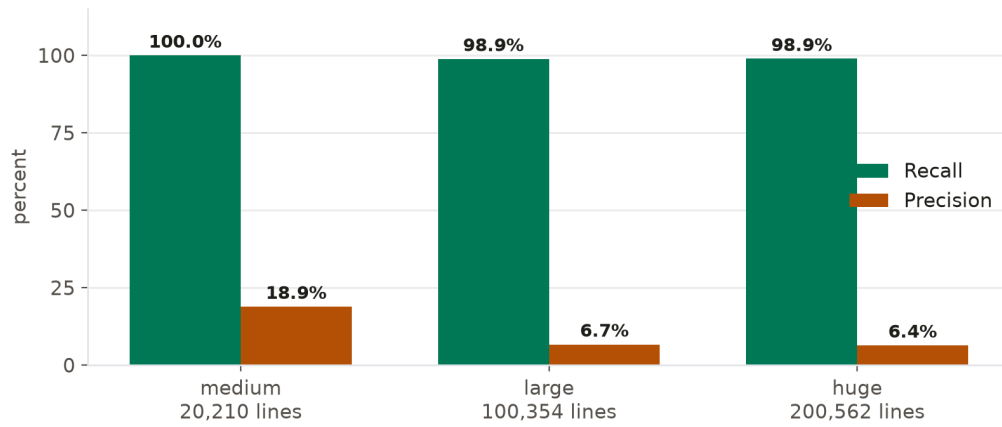
Trust claims are cheap; we measured ours. We built a **stress harness** that generates synthetic accounting populations with **known anomalies injected** — every planted anomaly is labeled with the rule that should catch it. Because the truth is known, recall and precision can be computed exactly.

Methodology

- Populations from **20,000 to 200,000+** lines, deterministic (same seed → same data).
- Bases made deliberately **realistic, not sterile**: ~90% manual + ~10% system-posted entries, legitimately rare users, legitimately large invoices, month-start postings — noise that a naive test would hide.
- Scoring per rule: **recall** = share of planted anomalies caught; **precision** = share of flags that were real anomalies; false positives counted only against clean documents.

Headline results

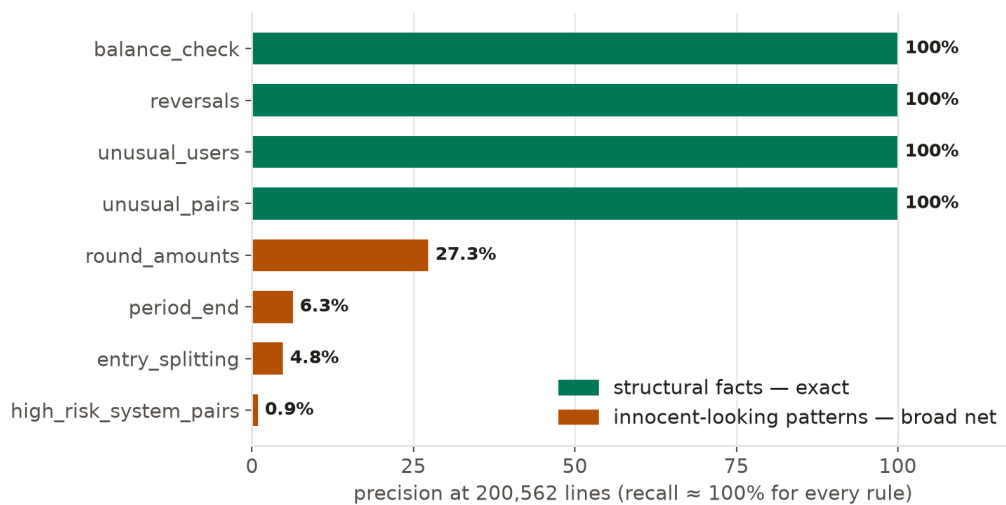
Scenario	Lines	Planted	Recall	Precision
medium	20,210	105	100%	18.9%
large	100,354	177	98.9%	6.7%
huge	200,562	281	98.9%	6.4%



Reading this honestly: the system *almost never misses a planted anomaly, at any scale* (200k lines ingested in ~14 seconds). But aggregate precision falls as populations grow — because benign patterns that *look* anomalous (a big round invoice, a month-start posting) grow with the population while true fraud stays rare.

Per-rule picture (200k-line scenario)

Rule	Recall	Precision
round_amounts	100%	27%
entry_splitting	100%	5%
period_end	100%	6%
unusual_pairs	96%	100%
unusual_users	100%	100%
balance_check	100%	100%
reversals	100%	100%
date_divergence	87%	100%
high_risk_system_pairs	100%	0.9%



Two clean stories emerge. Rules based on **structural facts** (balance checks, reversals, rare users, account pairs) are essentially perfect on both axes. Rules based on **patterns that have innocent explanations** (round amounts, timing, size clustering) catch everything but drown in legitimate look-alikes at scale.

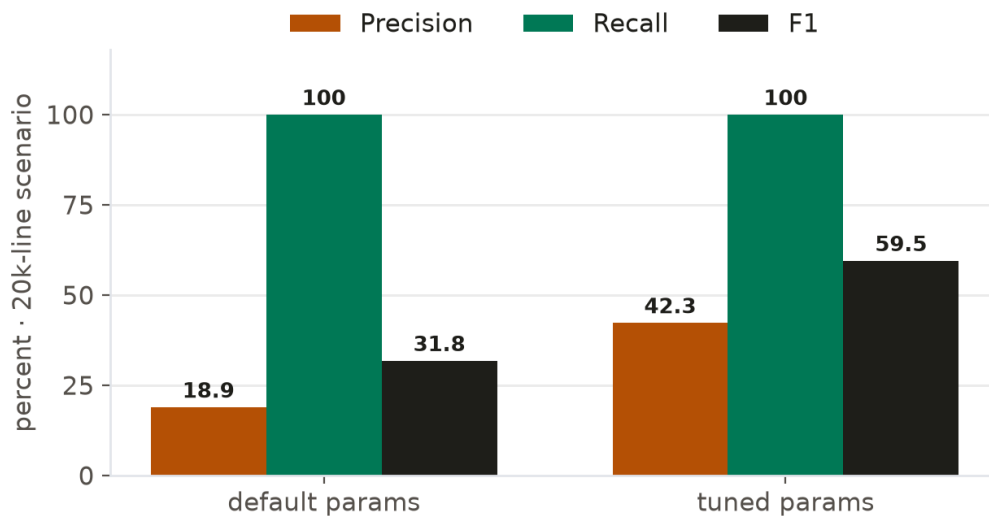
The universe check

Reviewers can only see a capped selection (the *universe*) ranked above performance materiality. In testing, **97–100% of above-materiality planted anomalies reached the reviewer** — the scoping logic does its job.

Can tuning help? Yes — measurably.

Using the same harness, two threshold changes (narrower period-end window, stricter rarity definition for system pairs) were tested A/B on the 20k scenario:

Metric	Default	Tuned
Aggregate precision	18.9%	42.3%
Aggregate recall	100%	100%



Precision more than doubled at zero recall cost — and because thresholds interact with the population, the lesson generalizes: **rule parameters must be configured against the engagement’s own materiality**, and this harness makes that calibration measurable instead of guesswork.

The AI layer

On the smallest scenario, the real LLM triage leg was scored separately: it **caught 100% of planted anomalies** but recommended inspecting many clean entries too (inspect precision $\approx 41\%$). That is precisely its role in the design: a broad prioritizer feeding the human, not a decider. The three layers trade roles exactly as intended — rules never miss, AI ranks everything, the human makes precision.

5 · Trust & Integrity

- **Hash-chained decisions** — every reviewer decision links cryptographically to the previous one; tampering is detectable.
- **Citation gates** — the AI narrative cannot be finalized unless every claim cites verifiable data from the run.
- **Deterministic core** — same input, same flags, always reproducible.
- **Human-only finalization** — four gates (review complete, procedures documented, citations valid, limitations stated) block the report until a human’s

decisions are in place.

- **Local-first** — engagement data stays on the machine; AI calls run in zero-retention privacy mode.

6 • Honest Limitations

- **Precision at scale is low by construction:** the rule layer is a sieve. This is a feature of the domain (fraud is rare and mimics normality), not a bug — but users must expect to review, not read raw flag counts.
- **One config-dependent rule** (`high_risk_system_pairs`) needs the auditor to define high-risk users; unconfigured, it fires indiscriminately (0.9% precision measured).
- `date_divergence` missed ~13% of plants whose fake document dates landed inside the benign window — a known edge, documented rather than hidden.
- The synthetic populations, however realistic, are still synthetic; field behavior on live client data will differ.

7 • Conclusion

JE Agent demonstrates that journal-entry testing can be **fast, transparent, and honest at once**: near-perfect detection of designed anomaly classes at 200k-line scale in seconds, an architecture that gives the AI no final authority, integrity guarantees that make the audit trail tamper-evident, and measured limitations published alongside the strengths. Its strongest credibility asset is the last one: **it**

tells you exactly what it is good at, what it is not, and shows the numbers for both.

Appendix A · The ten rules in plain language

#	Rule	What it looks for	Why it matters
1	manual_entries	entries typed by hand rather than posted by a system	manual entries are where override risk lives
2	round_amounts	amounts on suspiciously round numbers (10,000 / 100,000...)	invented figures are rarely precise
3	entry_splitting	one obligation broken into pieces just under approval limits	classic way to dodge a second signature
4	period_end	entries posted in the window around period close	closing pressure is when numbers get “managed”
5	unusual_pairs	account combinations that rarely occur together	e.g. revenue debited against an asset account
6	unusual_users	user IDs with almost no posting history acting alone	ghost or borrowed credentials leave little trace
7	balance_check	documents whose debits and credits don’t reconcile	unbalanced postings indicate broken controls
8	reversals	an entry reversed shortly after posting, same	reversal after benefit = possible

#	Rule	What it looks for	Why it matters
		amount	concealment
9	date_divergence	document date far from posting date	backdating shifts transactions into wrong periods
10	high_risk_system_pairs	configured high-risk users combined with unusual accounts	privileged users need scrutiny wherever they touch

Every rule is a transparent, inspectable check — no black boxes at this layer.

Appendix B • Running it yourself

JE Agent runs entirely on your machine. Three commands from a fresh clone:

```

1  git clone https://github.com/salaheddine-11/je-agent && cd je-agent
2  uv sync
3  uv run jeagent serve

```

Then open <http://localhost:8300> in a browser (default key: `jeagent`).

A typical engagement takes about five minutes of attention:

1. **New engagement** — drop in your CSV export; column mapping is detected automatically. Set overall/performance materiality, choose human or AI-assisted review mode, pick report language.
2. **Monitor** — watch the pipeline stages verify themselves: ingest → rules → cross-reference → triage.
3. **Review** — work the ledger. Every entry shows which rules fired and its full journal lines; each decision needs one click plus a written reason.

4. **Finalize** — four gates check review completeness, procedures, citations and limitations; then the PDF report, Excel worksheet and flagged-entry list are generated.

Requirements: any modern OS, Python 3.12+, ~200 MB free space. Internet is needed only for the optional AI triage calls (zero-retention mode); everything else runs locally, so client data never leaves the machine.

Glossary

Journal entry (JE) — the atomic record of an accounting transaction: dated lines, each crediting or debiting an account.

ISA 240 / AS 2401 — international (IAASB) and US (PCAOB) standards requiring auditors to address fraud risk, explicitly including journal-entry testing.

Materiality — the size threshold above which misstatements matter to the financial statements. JE Agent distinguishes *overall* materiality from *performance* materiality (the working threshold for testing).

Performance materiality (PM) — see above; entries above PM drive the review universe's scope.

Universe — the ranked selection of flagged entries the reviewer actually sees, scoped to items above performance materiality and capped for practical review.

Recall — of all true anomalies present, the share the system caught. Missing fraud is the costly error, so recall is the headline metric.

Precision — of everything the system flagged, the share that was genuinely anomalous. Low precision means noise the reviewer must wade through.

F1 — the balanced average of recall and precision.

Benford's law — natural data has predictable first-digit frequencies (1 is far more common than 9). Fabricated numbers drift away from this pattern; JE Agent reports the comparison as an informational indicator only.

Hash chain — each decision record carries a cryptographic fingerprint of the previous one, so altering history breaks the chain and becomes detectable.

Zero retention — AI calls configured so the provider stores nothing about the request contents.